

SciVault Question Format

Human-Readable Specification

SciVault-QF Version 0.1

SciVault

Abstract

The SciVault Question Format (SciVault-QF) is an open, application-independent format for representing and exchanging educational questions and question banks. It provides a common question-content layer for the SciVault software suite and may also be implemented by third-party applications.

SciVault-QF separates question content from presentation, assessment sessions, student records, and results. This allows a question to be authored once and reused for test generation, peer instruction, practice, homework, online assessment, analytics, and other instructional purposes.

Contents

1	Scope	3
2	Design Goals	3
3	Format Identity	3
4	File Representation	4
5	SciVault-QF Package Format	4
5.1	Package Contents	4
5.2	Package Manifest	5
5.3	Package Paths	5
5.4	Packaged Media	5
6	Document Structure	6
7	Question-Set Metadata	6
8	Identifiers	7
9	Common Question Structure	8
10	Text and Mathematical Notation	8
11	Difficulty	9
12	Tags and Learning Objectives	9

13 Multiple-Choice Questions	10
14 Multiple-Select Questions	10
15 True/False Questions	11
16 Short-Answer Questions	11
17 Fill-in-the-Blank Questions	12
18 Numerical Questions	12
19 Hints and Explanations	13
20 Media	13
21 Dynamic Questions	14
21.1 Integer Variables	14
21.2 Decimal Variables	14
21.3 Choice Variables	14
22 Dynamic Expressions	15
23 Randomization	15
24 Extensions	15
25 Separation of Content and Operational Data	16
26 Validation	16
26.1 Validation Severity	16
26.2 Structural Errors	17
26.3 Semantic Errors	17
26.4 Warnings	17
26.5 Validation Findings	18
27 Unsupported Features	18
28 Accessibility	19
29 Licensing	19
30 Question Types Defined by Version 0.1	19
31 Future Considerations	19
32 Complete Example	20
33 Conformance	21
34 Summary	21

1 Scope

SciVault-QF defines the representation of educational questions and collections of questions.

It is intended to support applications including:

- question-bank editors;
- test and worksheet generators;
- peer-instruction systems;
- classroom-response systems;
- homework and practice systems;
- online assessments;
- item-analysis systems;
- learning analytics;
- learning-management-system integrations; and
- compatible third-party educational software.

SciVault-QF does **not** define student accounts, classroom rosters, assessment sessions, grades, or student responses.

2 Design Goals

SciVault-QF is designed to be:

- human-readable;
- machine-readable;
- portable;
- extensible;
- suitable for version control;
- independent of any particular database;
- independent of any particular SciVault application;
- suitable for mathematical and scientific content;
- capable of referencing media;
- suitable for static and dynamically generated questions; and
- usable by software outside the SciVault suite.

3 Format Identity

The official name of the format is:

SciVault Question Format

The abbreviated machine-readable identifier is:

```
SciVault-QF
```

Version 0.1 files shall contain:

```
"format": "SciVault-QF",  
"format_version": "0.1"
```

4 File Representation

SciVault-QF version 0.1 defines both a standalone JSON document representation and a packaged question-set representation.

A standalone SciVault-QF document uses:

- **Encoding:** UTF-8
- **Extension:** `.json`
- **Recommended MIME type:** `application/json`

Standalone JSON documents are suitable for development, validation, version control, inter-change, and applications that do not require bundled media.

A packaged SciVault-QF question set uses the package format defined in the following section. The package extension is `.sqf`.

5 SciVault-QF Package Format

A SciVault-QF package is a ZIP archive containing exactly one SciVault-QF question set together with any locally distributed resources required by that question set.

- **Extension:** `.sqf`
- **Container format:** ZIP
- **Text encoding:** UTF-8
- **Path separator:** `/`

The `.sqf` extension identifies the file as a SciVault-QF package. Applications shall interpret the file as a ZIP container conforming to the rules in this section.

5.1 Package Contents

A minimal package contains:

```
example-set.sqf
|-- manifest.json
'-- questions.json
```

A package containing local resources may contain:

```
example-set.sqf
|-- manifest.json
|-- questions.json
'-- media/
    |-- diagram.svg
    '-- setup-photo.jpg
```

The package shall contain exactly one SciVault-QF question set.

The file `manifest.json` shall appear at the root of the archive. The SciVault-QF document referenced by the manifest shall also be contained within the archive.

The `media/` directory is optional. Files and subdirectories within `media/` may be organized as appropriate for the question set. Applications shall use media references in the SciVault-QF document rather than infer resource meaning from directory or file names.

5.2 Package Manifest

The package manifest identifies the package format and the SciVault-QF document contained in the package.

A version 0.1 manifest has the following form:

```
{
  "format": "SciVault-QF",
  "format_version": "0.1",
  "content": "questions.json"
}
```

Field	Type	Required	Description
<code>format</code>	String	Yes	Must contain <code>SciVault-QF</code> .
<code>format_version</code>	String	Yes	Version of the SciVault-QF package format.
<code>content</code>	String	Yes	Package-relative path to the SciVault-QF JSON document.

Version 0.1 package manifests shall contain only the fields defined above. Additional properties are not permitted.

Version 0.1 packages should normally use `questions.json` as the content filename. Readers shall use the `content` field rather than assume a fixed filename.

Question-set title, author, license, tags, and similar content metadata shall remain in the SciVault-QF document and shall not be duplicated in the package manifest.

5.3 Package Paths

Paths stored in a SciVault-QF package are relative to the package root. Forward slashes shall be used as path separators regardless of the host operating system.

Package paths:

- shall not be absolute;
- shall not begin with `/`;
- shall not contain backslashes;
- shall not contain an empty path component;
- shall not contain a path component equal to `.`;
- shall not contain a path component equal to `..`; and
- shall identify a unique archive entry.

Every archive entry path shall satisfy these requirements, including entries that are not referenced by the package manifest or SciVault-QF document.

Duplicate ZIP entry names are invalid.

Path names are case-sensitive for purposes of SciVault-QF package resolution, even when a package is read on a case-insensitive file system.

Readers that extract package contents shall prevent archive entries from being written outside the intended extraction directory.

5.4 Packaged Media

A local media reference in a packaged question set shall use a package-root-relative path.

Example:

```
"src": "media/images/force-diagram.svg"
```

This path refers to the archive entry:

```
media/images/force-diagram.svg
```

Subdirectories within `media/` are permitted. Their organization is not semantically significant; the `src` field is the authoritative location of the referenced resource.

External media references, such as HTTPS URLs, are not package resources and need not correspond to archive entries. Package validation shall require only local media references to resolve to entries contained in the package.

6 Document Structure

A SciVault-QF document contains metadata and one or more questions.

```
{
  "format": "SciVault-QF",
  "format_version": "0.1",

  "metadata": {
    ...
  },

  "questions": [
    ...
  ]
}
```

The top-level fields are:

Field	Type	Required	Description
<code>format</code>	String	Yes	Must contain <code>SciVault-QF</code> .
<code>format_version</code>	String	Yes	Version of this specification.
<code>metadata</code>	Object	Yes	Metadata describing the question collection.
<code>questions</code>	Array	Yes	Array containing one or more questions.
<code>extensions</code>	Object	No	Namespaced extension data.

Unknown fields shall not be inserted directly into core SciVault-QF objects. Additional data shall use the extension mechanism described later in this specification.

7 Question-Set Metadata

Example:

```
"metadata": {
  "id": "org.scivault.physics.kinematics",
```

```

    "title": "Kinematics Concept Questions",
    "description":
    "Conceptual and quantitative introductory kinematics questions.",
    "author": "Jane Doe",
    "license": "CC-BY-4.0",
    "version": "1.0",
    "language": "en-US",
    "tags": [
    "physics",
    "kinematics"
    ],
    "created": "2026-08-19T10:30:00-06:00",
    "modified": "2026-08-19T10:45:00-06:00"
  }

```

Field	Type	Required	Description
id	String	Recommended	Stable identifier for the question collection.
title	String	Yes	Human-readable title.
description	String	No	Description of the collection.
author	String	No	Author or responsible organization.
license	String	Yes	License governing the question content.
version	String	No	Version of the question collection.
language	String	No	Content language or locale, preferably expressed using a BCP 47 tag such as en-US .
tags	Array	No	Descriptive keywords applying to the collection.
created	String	No	Creation timestamp in ISO 8601 date-time format.
modified	String	No	Modification timestamp in ISO 8601 date-time format.
extensions	Object	No	Namespaced extension data.

The number of questions shall not be stored separately. Applications shall determine it from the length of the `questions` array.

8 Identifiers

Each question shall have an ID unique within its question collection.

Example:

```
"id": "q001"
```

A stable namespace-style identifier is recommended for the question set:

```
"id": "org.scivault.physics.kinematics"
```

The combination:

org.scivault.physics.kinematics:q001

may therefore be treated as a globally meaningful reference to the question.

Applications shall not assume that IDs are numeric or sequential.

Answer choices shall also have stable IDs unique within their question.

9 Common Question Structure

All questions shall contain:

- `id`;
- `type`;
- `question`; and
- `answer`.

Questions may additionally contain:

- `choices`;
- `explanation`;
- `hint`;
- `difficulty`;
- `tags`;
- `objectives`;
- `media`;
- `variables`;
- `settings`; and
- `extensions`.

Example:

```
{
  "id": "q001",
  "type": "multiple_choice",
  "question": "What is  $2+2$ ?",
  "choices": [
    {"id": "A", "text": "3"},
    {"id": "B", "text": "4"},
    {"id": "C", "text": "5"}
  ],
  "answer": "B",
  "explanation": " $2+2=4$ .",
  "difficulty": 1,
  "tags": ["math", "arithmetic"]
}
```

10 Text and Mathematical Notation

Question prompts, choices, hints, explanations, and other textual content may contain LaTeX-compatible mathematical notation.

Example:

```
"What is the kinetic energy of a  $4\,\mathrm{kg}$  object  
moving at  $3\,\mathrm{m/s}$ ?"
```

Because backslashes have special meaning in JSON, LaTeX backslashes shall be escaped.
For example:

```
\frac{1}{2}mv^2
```

is represented within a JSON string as:

```
"\\frac{1}{2}mv^2"
```

Applications should use a mathematical rendering system such as MathJax or KaTeX.

11 Difficulty

The optional `difficulty` field shall contain an integer from 0 through 10 inclusive.

The scale is intended to permit question authors and applications to classify questions with greater resolution than a small categorical scale.

The endpoints may generally be interpreted as:

0	Trivial or introductory
5	Moderate
10	Extremely difficult

Intermediate values represent increasing levels of difficulty.

Difficulty is primarily intended for:

- filtering;
- question-bank organization;
- automatic test generation;
- differentiated practice;
- adaptive systems; and
- instructional planning.

A difficulty value is an author or system estimate unless supported by empirical item statistics.

12 Tags and Learning Objectives

Tags provide descriptive classification.

Example:

```
"tags": [  
  "physics",  
  "kinematics",  
  "constant-acceleration"  
]
```

Learning objectives are stored separately:

```
"objectives": [
  "KIN-VELOCITY",
  "PHY.KIN.3"
]
```

This distinction permits questions to be searched broadly by topic while also associating them with specific assessable outcomes.

SciVault-QF does not require a particular learning-objective or standards framework.

13 Multiple-Choice Questions

Multiple-choice questions use:

```
"type": "multiple_choice"
```

They shall contain two or more choices and exactly one correct answer.

```
{
  "id": "q001",
  "type": "multiple_choice",

  "question":
    "An object starts from rest and accelerates at "
    "$2\\,\\mathrm{m/s^2}$ for $5\\,\\mathrm{s}$. "
    "What is its final velocity?",

  "choices": [
    {"id": "A", "text": "$5\\,\\mathrm{m/s}$"},
    {"id": "B", "text": "$10\\,\\mathrm{m/s}$"},
    {"id": "C", "text": "$15\\,\\mathrm{m/s}$"},
    {"id": "D", "text": "$20\\,\\mathrm{m/s}$"}
  ],

  "answer": "B",

  "explanation":
    "Using $v=v_0+at$, "
    "$v=0+(2)(5)=10\\,\\mathrm{m/s}$.",

  "difficulty": 2
}
```

The **answer** shall reference the stable ID of the correct choice. Displayed answer text shall not serve as the answer identifier.

14 Multiple-Select Questions

Questions having more than one correct choice use:

```
"type": "multiple_select"
```

The answer shall contain an array of correct choice IDs.

```
{
  "id": "q002",
  "type": "multiple_select",
  "question": "Which of the following are vector quantities?",
  "choices": [
    {"id": "A", "text": "Velocity"},
    {"id": "B", "text": "Mass"},
    {"id": "C", "text": "Acceleration"},
    {"id": "D", "text": "Temperature"}
  ],
  "answer": ["A", "C"]
}
```

15 True/False Questions

True/false questions use:

```
"type": "true_false"
```

They shall contain exactly two choices with the IDs:

```
true
false
```

Example:

```
{
  "id": "q003",
  "type": "true_false",
  "question": "An object moving at constant velocity has zero acceleration.",
  "choices": [
    {"id": "true", "text": "True"},
    {"id": "false", "text": "False"}
  ],
  "answer": "true"
}
```

16 Short-Answer Questions

Short-answer questions use:

```
"type": "short_answer"
```

The answer may contain one acceptable response:

```
"answer": "photosynthesis"
```

or multiple acceptable textual responses:

```
"answer": [  
  "photosynthesis",  
  "Photosynthesis"  
]
```

Applications may implement normalization, case-insensitive comparison, teacher review, or other appropriate grading behavior.

17 Fill-in-the-Blank Questions

Fill-in-the-blank questions use:

```
"type": "fill_in_the_blank"
```

Version 0.1 reserves:

```
{{blank}}
```

as the blank placeholder.

Example:

```
{  
  "id": "q004",  
  "type": "fill_in_the_blank",  
  "question": "The SI unit of force is the {{blank}}.",  
  "answer": ["newton", "Newton", "N"]  
}
```

Version 0.1 defines one scored blank per question. Multiple independently scored blanks are reserved for a future revision.

18 Numerical Questions

Numerical questions use:

```
"type": "numerical"
```

A static numerical answer may contain:

```
"answer": {  
  "value": 20,  
  "tolerance": 0.1,  
  "unit": "m/s"  
}
```

The **value** is the expected answer. The optional **tolerance** specifies an absolute numerical tolerance. The optional **unit** specifies the expected physical unit.

A dynamically calculated answer may instead use an expression:

```
"answer": {
  "expression": "a * t",
  "tolerance": 0.01,
  "unit": "m/s"
}
```

Exactly one of **value** or **expression** shall be present.

19 Hints and Explanations

The optional **hint** field contains assistance that may be displayed before the answer is revealed.

Example:

```
"hint": "Identify the initial velocity, acceleration, and elapsed time."
```

The optional **explanation** field contains feedback, reasoning, or a worked solution.

Example:

```
"explanation":
"Using  $v=v_0+at$ , "
" $v=0+(2)(5)=10$ \\,\\mathrm{m/s}$."
```

Consuming applications determine when hints and explanations are displayed.

20 Media

Questions and answer choices may reference external media.

Example:

```
"media": [
{
  "id": "diagram1",
  "type": "image",
  "src": "media/incline.svg",
  "alt": "A block resting on a 30 degree inclined plane."
}
]
```

Version 0.1 recognizes the media types:

- **image**;
- **audio**;
- **video**.

Media objects may contain:

- **id**;
- **type**;

- `src`;
- `alt`;
- `caption`;
- `credit`;
- `license`; and
- `extensions`.

Images shall contain an `alt` field.

Relative paths are recommended for locally distributed resources. In a SciVault-QF package, local `src` values shall be package-root-relative paths as defined by the SciVault-QF Package Format section.

21 Dynamic Questions

Dynamic questions may define variables whose values are generated when the question is instantiated.

Variables are referenced using double braces.

Example:

```
"question":
  "An object accelerates at {{a}} m/s^2 "
  "for {{t}} s. What is its change in velocity?"
```

Version 0.1 defines:

- integer variables;
- decimal variables;
- choice variables.

21.1 Integer Variables

```
"a": {"type": "integer", "min": 1, "max": 5}
```

An optional positive `step` may be specified.

21.2 Decimal Variables

```
"mu": {"type": "decimal", "min": 0.1, "max": 1.0, "step": 0.1}
```

21.3 Choice Variables

```
"planet": {
  "type": "choice",
  "values": ["Earth", "Mars", "Moon"]
}
```

Choice-variable values may be strings, numbers, or Boolean values.

22 Dynamic Expressions

A dynamic numerical answer may be calculated from variables.

Example:

```
{
  "id": "q005",
  "type": "numerical",
  "question":
    "An object starts from rest and accelerates at "
    "{{a}} m/s^2 for {{t}} s. What is its final speed?",
  "variables": {
    "a": {"type": "integer", "min": 1, "max": 5},
    "t": {"type": "integer", "min": 2, "max": 10}
  },
  "answer": {
    "expression": "a * t",
    "tolerance": 0.01,
    "unit": "m/s"
  }
}
```

Expressions shall be evaluated by a restricted mathematical expression engine.

SciVault-QF files shall never cause arbitrary programming-language code to be executed.

The complete expression language is reserved for further specification.

23 Randomization

Applications may randomize:

- question selection;
- question order;
- dynamic variable values; and
- answer-choice order.

Correctness is associated with stable choice IDs rather than displayed positions.

A question may recommend answer-choice shuffling:

```
"settings": {
  "shuffle_choices": true
}
```

24 Extensions

SciVault-QF is extensible while maintaining strict core validation.

Application-specific data shall be placed inside an **extensions** object.

Example:

```
"extensions": {
  "org.scivault.peer_instruction": {
    "recommended_discussion_time": 120
  }
}
```

```
}  
}
```

Reverse-domain namespaces are recommended to reduce naming collisions. Applications should preserve extension namespaces they do not understand whenever practical. An application may ignore unsupported extension data. Core question correctness should not depend upon proprietary extension data when portability is intended.

25 Separation of Content and Operational Data

SciVault-QF represents question content.

It shall not contain identifiable student or session information such as:

- student names;
- student IDs;
- class rosters;
- individual responses;
- grades;
- attendance;
- assessment sessions;
- peer-instruction rounds; or
- response timestamps.

Applications shall store operational data separately and reference SciVault-QF questions using stable identifiers.

26 Validation

SciVault-QF document validation consists of:

1. structural validation using the official JSON Schema;
2. semantic validation using a SciVault-QF validator.

The official schema uses JSON Schema Draft 2020-12.

Package validation additionally verifies the package manifest, archive paths, contained document, and local resource references.

Package validation and SciVault-QF document validation are logically distinct. A conforming package nevertheless requires the SciVault-QF document referenced by its manifest to conform to SciVault-QF 0.1.

Resources contained in a package but not referenced by the SciVault-QF document may produce a warning but do not by themselves make the package invalid.

26.1 Validation Severity

Findings have one of three severities.

error	The content violates a requirement or cannot be interpreted reliably. One or more errors make the document or package invalid.
warning	The content remains valid but contains a potentially incomplete, ambiguous, undesirable, or unusual condition.
info	Informational feedback that does not indicate a defect.

Warnings do not make a document or package invalid.

26.2 Structural Errors

The official JSON Schema validates:

- required fields;
- field data types;
- question types;
- answer structures;
- difficulty values;
- media structures;
- dynamic-variable structures;
- ISO 8601 timestamps; and
- unknown core fields.

JSON Schema validation failures are errors.

26.3 Semantic Errors

The semantic validator shall detect conditions including:

- duplicate question IDs;
- duplicate choice IDs;
- answers referencing nonexistent choices;
- malformed true/false choices;
- variable minimum greater than maximum;
- undefined variable placeholders;
- undefined variables in expressions;
- forbidden expression syntax;
- missing required local media;
- fill-in-the-blank questions without `{{blank}}`; and
- violations of mandatory identifier rules.

26.4 Warnings

Warnings may include:

- missing explanation;
- missing difficulty;
- missing tags;
- missing objectives;
- inadequate image alternative text;
- duplicate displayed choice text;
- suspiciously similar answer choices;
- unusually long question content;
- unused dynamic variables;
- unused media;
- missing question-set ID;
- missing question-set version; and
- nonstandard extension namespaces.

26.5 Validation Findings

Validators should return structured findings.

Example:

```
{
  "severity": "error",
  "code": "ANSWER_UNKNOWN_CHOICE",
  "message": "Answer 'E' does not reference an existing choice.",
  "question_id": "q017",
  "path": "questions[16].answer"
}
```

Recommended fields are:

- **severity**;
- **code**;
- **message**;
- **question_id**, when applicable;
- **path**, when applicable.

Stable machine-readable validation codes should be used.

Initial codes include:

```
DUPLICATE_QUESTION_ID
DUPLICATE_CHOICE_ID
ANSWER_UNKNOWN_CHOICE
UNDEFINED_VARIABLE
INVALID_VARIABLE_RANGE
INVALID_EXPRESSION
MISSING_MEDIA
MISSING_BLANK
MISSING_EXPLANATION
MISSING_OBJECTIVES
DUPLICATE_CHOICE_TEXT
UNUSED_VARIABLE
UNUSED_MEDIA
```

27 Unsupported Features

A SciVault application is not required to implement every question type or feature defined by SciVault-QF.

When encountering valid content it does not support, an application should:

1. preserve the content;
2. identify it as unsupported;
3. avoid corrupting or deleting it; and
4. continue processing supported content whenever possible.

For example, an early version of SciVault Peer Instruction may support only **multiple_choice** even though the question bank contains other valid SciVault-QF question types.

28 Accessibility

Question authors and applications should consider accessibility throughout the content lifecycle.

At minimum:

- meaningful images shall contain alternative text;
- correctness shall not depend solely on color;
- mathematical expressions should be represented semantically when practical;
- questions should remain usable with assistive technology when practical; and
- descriptions should communicate media information required to answer a question.

29 Licensing

Each question collection shall specify a content license.

Example:

```
"license": "CC-BY-4.0"
```

The license governing SciVault software is independent of the license governing SciVault-QF content.

Individual media objects may specify different licenses where necessary.

30 Question Types Defined by Version 0.1

<code>multiple_choice</code>	Exactly one correct answer selected from discrete choices.
<code>multiple_select</code>	One or more correct answers selected from discrete choices.
<code>true_false</code>	Binary true/false question.
<code>short_answer</code>	Short textual response.
<code>fill_in_the_blank</code>	One textual blank contained within a prompt.
<code>numerical</code>	Numerical response with optional tolerance and physical unit.

31 Future Considerations

Potential future features include:

- multipart questions;
- multiple independently scored blanks;
- ranking;
- matching;
- ordering;
- graphical responses;
- image-region selection;
- vector drawing;
- symbolic equation entry;
- extended-response questions;
- passages and question groups;
- random question pools;

- conditional questions;
- significant-figure checking;
- physical-unit equivalence;
- richer dynamic-expression syntax;
- embedded SVG diagrams;
- distractor misconception metadata;
- empirical item-difficulty statistics;
- discrimination indices;
- standards-framework references;
- localization and translated variants; and
- an expanded standard validation-code registry.

32 Complete Example

```
{
  "format": "SciVault-QF",
  "format_version": "0.1",
  "metadata": {
    "id": "org.scivault.physics.kinematics",
    "title": "Kinematics Concept Questions",
    "description": "Introductory kinematics questions.",
    "author": "Jane Doe",
    "license": "CC-BY-4.0",
    "version": "1.0",
    "language": "en-US",
    "tags": ["physics", "kinematics"]
  },
  "questions": [
    {
      "id": "q001",
      "type": "multiple_choice",
      "question":
        "An object starts from rest and accelerates at "
        "$2\\,\\mathrm{m/s^2}$ for $5\\,\\mathrm{s}$. "
        "What is its final velocity?",
      "choices": [
        {"id": "A", "text": "$5\\,\\mathrm{m/s}$"},
        {"id": "B", "text": "$10\\,\\mathrm{m/s}$"},
        {"id": "C", "text": "$15\\,\\mathrm{m/s}$"},
        {"id": "D", "text": "$20\\,\\mathrm{m/s}$"}
      ],
      "answer": "B",
      "explanation":
        "Using $v=v_0+at$, $v=0+(2)(5)=10\\,\\mathrm{m/s}$.",
      "difficulty": 2,
      "tags": ["physics", "kinematics", "constant-acceleration"],
      "objectives": ["KIN-VELOCITY"],
      "settings": {"shuffle_choices": true}
    }
  ]
}
```

33 Conformance

A standalone document conforms to SciVault-QF 0.1 when:

1. it is valid JSON encoded as UTF-8;
2. it identifies itself as **SciVault-QF**;
3. its `format_version` is `0.1`;
4. it passes the official SciVault-QF 0.1 JSON Schema; and
5. it contains no errors identified by the normative SciVault-QF semantic validation rules.

A SciVault-QF package conforms to version 0.1 when:

1. it is a valid ZIP archive using the `.sqf` extension;
2. it contains a valid root-level `manifest.json`;
3. the manifest identifies **SciVault-QF** version `0.1`;
4. the manifest references exactly one contained SciVault-QF document;
5. the referenced document conforms to SciVault-QF 0.1;
6. all required local media references resolve to valid package entries;
7. package paths satisfy the package path requirements; and
8. it contains no package-level semantic validation errors.

Warnings and informational findings do not prevent conformance.

34 Summary

SciVault-QF provides the common question-content representation for the SciVault software ecosystem.

Version 0.1 supports standalone UTF-8 JSON documents using the standard `.json` extension and portable ZIP-based question-set packages using the `.sqf` extension.

Question and answer-choice IDs are stable identifiers. Question content is kept separate from student and session data.

SciVault-QF supports scientific and mathematical notation, media, difficulty metadata, learning objectives, dynamic variables, several question types, validation, and namespaced extensions.

SciVault-QF packages may contain one question set and its associated local resources. Future specifications may extend the question model and package capabilities while preserving portable interchange.